



SW Pilot 학습보고서 <과정 10>

파이썬 5팀 황문규

문제 1. 토마토의 작황을 알아보자 - 코드

```
1 # 과정 10 - (문제1) 토마토의 작황을 알아보자
2 import matplotlib.pyplot as plt
3 from sklearn.model_selection import train_test_split
4 from sklearn.neighbors import KNeighborsRegressor
5 from sklearn.metrics import mean_absolute_error, mean_squared_error
6 import numpy as np
7 import mglearn
8
9 def evaluate_model(model, x_test, y_test):
10     y_pred = model.predict(x_test)
11     r2 = model.score(x_test, y_test)
12     mae = mean_absolute_error(y_test, y_pred)
13     rmse = np.sqrt(mean_squared_error(y_test, y_pred))
14     return r2, mae, rmse
15
16 def main():
17     # 데이터 생성
18     x, y = mglearn.datasets.make_wave(n_samples=40)
19
20     # 학습/테스트 분할
21     x_train, x_test, y_train, y_test = train_test_split(
22         x, y, test_size=0.25, random_state=0
23     )
24
25     # 결과 저장용
26     results = {}
27
28     # n_neighbors = 1, 3, 9 반복 학습 및 평가
29     for k in [1, 3, 9]:
30         model = KNeighborsRegressor(n_neighbors=k)
31         model.fit(x_train, y_train)
32
33         # [보너스 과제] - MAE, RMSE 값 확인
34         r2, mae, rmse = evaluate_model(model, x_test, y_test)
35         results[k] = {
36             'r2': r2,
37             'mae': mae,
38             'rmse': rmse
39         }
40
41         print(f'\nn_neighbors = {k}'])
42         print(f'R² : {r2:.4f}'])
43         print(f'MAE: {mae:.4f}'])
44         print(f'RMSE: {rmse:.4f}'])
45
46     # [보너스 과제] - 가장 성능이 좋은 모델 찾기 (기준: R²가 가장 높고 RMSE가 가장 낮은 모델)
47     best_k = max(results, key=lambda k: (results[k]['r2'], -results[k]['rmse']))
48     print(f'\n✅ 가장 성능이 좋은 모델: n_neighbors = {best_k}'])
49
50
51 if __name__ == '__main__':
52     main()
```

문제 1. 토마토의 작황을 알아보자 - 구현화면

```
swpilotPyhon5 — -zsh — 100x30
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process10/10-1/main.py ]

[n_neighbors = 1]
R² : 0.3520
MAE: 0.5823
RMSE: 0.6522

[n_neighbors = 3]
R² : 0.8344
MAE: 0.2537
RMSE: 0.3297

[n_neighbors = 9]
R² : 0.6541
MAE: 0.3667
RMSE: 0.4765

✅ 가장 성능이 좋은 모델 : n_neighbors = 3
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % ]
```

문제 2. 토마토의 출하량을 예측해 보자 - 코드

```
1 # 과정 10 - (문제2) 토마토의 출하량을 예측해 보자
2 import mglearn
3 from sklearn.linear_model import LinearRegression, Ridge
4 from sklearn.model_selection import train_test_split
5
6 def main():
7     x, y = mglearn.datasets.make_wave(n_samples=60)
8
9     x_train, x_test, y_train, y_test = train_test_split(
10         x, y, random_state=42
11     )
12
13     model = LinearRegression()
14     model.fit(x_train, y_train)
15
16     print('\n-----선형 회귀 결과-----')
17     print('가중치:', model.coef_)
18     print('절편:', model.intercept_)
19     print('훈련 세트 점수:', model.score(x_train, y_train))
20     print('테스트 세트 점수:', model.score(x_test, y_test))
21
22     # [보너스 과제] - Ridge 알고리즘 사용
23     print('\n-----릿지 회귀 결과-----')
24     for alpha in [i / 10 for i in range(1, 11)]:
25         ridge = Ridge(alpha=alpha)
26         ridge.fit(x_train, y_train)
27         print(f'alpha = {alpha:.1f}')
28         print('가중치:', ridge.coef_)
29         print('절편:', ridge.intercept_)
30         print('훈련 세트 점수:', ridge.score(x_train, y_train))
31         print('테스트 세트 점수:', ridge.score(x_test, y_test))
32
33 if __name__ == '__main__':
34     main()
```

문제 2. 토마토의 출하량을 예측해 보자 - 구현화면

```
swpilotPyhon5 --zsh-- 162x64
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process10/10-2/main.py

-----선형 회귀 결과-----
가중치 : [0.39390555]
절편 : -0.031804343026759704
훈련 세트 점수 : 0.6700890315075756
테스트 세트 점수 : 0.6593368596863702

-----릿지 회귀 결과-----
alpha = 0.1
가중치 : [0.39364443]
절편 : -0.03188343931569497
훈련 세트 점수 : 0.6700887370524764
테스트 세트 점수 : 0.6591834089791664
alpha = 0.2
가중치 : [0.39338366]
절편 : -0.03196243080938144
훈련 세트 점수 : 0.670087855247165
테스트 세트 점수 : 0.6590297850228881
alpha = 0.3
가중치 : [0.39312324]
절편 : -0.032041317715947176
훈련 세트 점수 : 0.6700863884246516
테스트 세트 점수 : 0.6588759889089801
alpha = 0.4
가중치 : [0.39286316]
절편 : -0.0321201002429696
훈련 세트 점수 : 0.6700843389086875
테스트 세트 점수 : 0.6587220217240226
alpha = 0.5
가중치 : [0.39260342]
절편 : -0.032198778597476926
훈련 세트 점수 : 0.6700817090138063
테스트 세트 점수 : 0.6585678845497533
alpha = 0.6
가중치 : [0.39234402]
절편 : -0.03227735298595061
훈련 세트 점수 : 0.6700785010453643
테스트 세트 점수 : 0.6584135784630902
alpha = 0.7
가중치 : [0.39208497]
절편 : -0.0323558236143264
훈련 세트 점수 : 0.6700747172995807
테스트 세트 점수 : 0.6582591045361545
alpha = 0.8
가중치 : [0.39182426]
절편 : -0.03243419068799683
훈련 세트 점수 : 0.6700703600635779
테스트 세트 점수 : 0.6581044638362918
alpha = 0.9
가중치 : [0.39156789]
절편 : -0.032512454411812386
훈련 세트 점수 : 0.6700654316154213
테스트 세트 점수 : 0.6579496574260957
alpha = 1.0
가중치 : [0.39130987]
절편 : -0.03259061499008391
훈련 세트 점수 : 0.6700599342241599
테스트 세트 점수 : 0.6577946863634279
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```

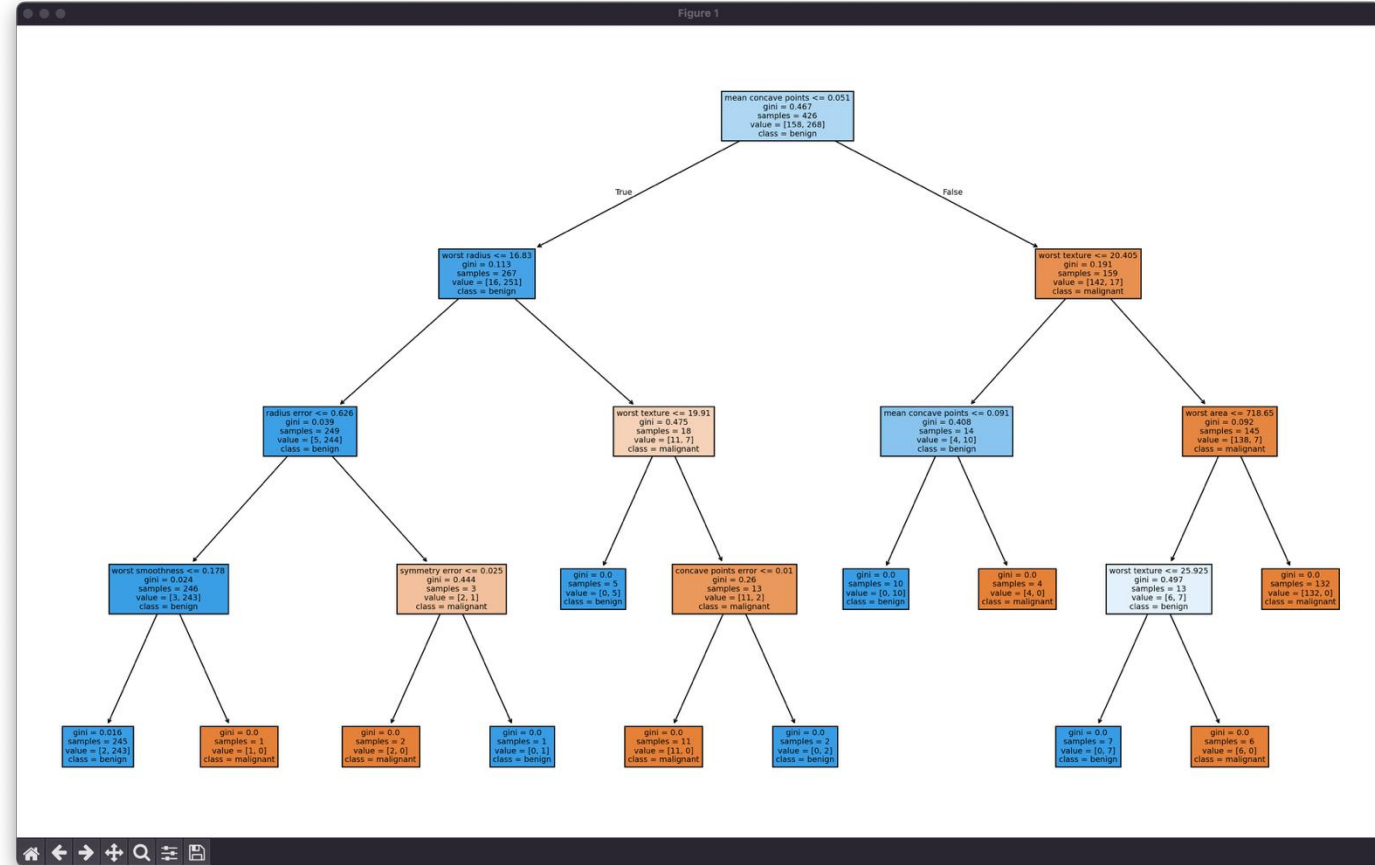
문제 3. 뜻밖의 발암물질들... - 코드

```
1 # 과정 10 - (문제3) 뜻밖의 발암물질들...
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import train_test_split
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.tree import plot_tree
6 import matplotlib.pyplot as plt
7
8 def main():
9     # 데이터 로드
10    data = load_breast_cancer()
11    x = data.data
12    y = data.target
13
14    # 데이터 분할
15    x_train, x_test, y_train, y_test = train_test_split(
16        x, y, random_state=42
17    )
18
19    # 첫 번째 실험: 기본 DecisionTreeClassifier
20    model1 = DecisionTreeClassifier()
21    model1.fit(x_train, y_train)
22    print('기본 모델 결과')
23    print('훈련 정확도:', model1.score(x_train, y_train))
24    print('테스트 정확도:', model1.score(x_test, y_test))
25
26    # 두 번째 실험: max_depth=4, random_state=0 지정
27    model2 = DecisionTreeClassifier(max_depth=4, random_state=0)
28    model2.fit(x_train, y_train)
29    print('\nmax_depth=4 모델 결과')
30    print('훈련 정확도:', model2.score(x_train, y_train))
31    print('테스트 정확도:', model2.score(x_test, y_test))
32
33    # [보너스 과제] 결정 트리 시각화
34    plt.figure(figsize=(20, 10))
35    plot_tree(model2, filled=True,
36              feature_names=data.feature_names,
37              class_names=data.target_names)
38    plt.tight_layout()
39    plt.show()
40
41
42 if __name__ == '__main__':
43     main()
```

문제 3. 뜻밖의 발암물질들... - 구현화면

```
swpilotPyhon5 — python src/Process10/10-3/main.py — 99x28
[[swpilot] hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process10/10-3/main.py
기본 모델 결과
훈련 정확도 : 1.0
테스트 정확도 : 0.9230769230769231

max_depth=4 모델 결과
훈련 정확도 : 0.9953051643192489
테스트 정확도 : 0.951048951048951
```

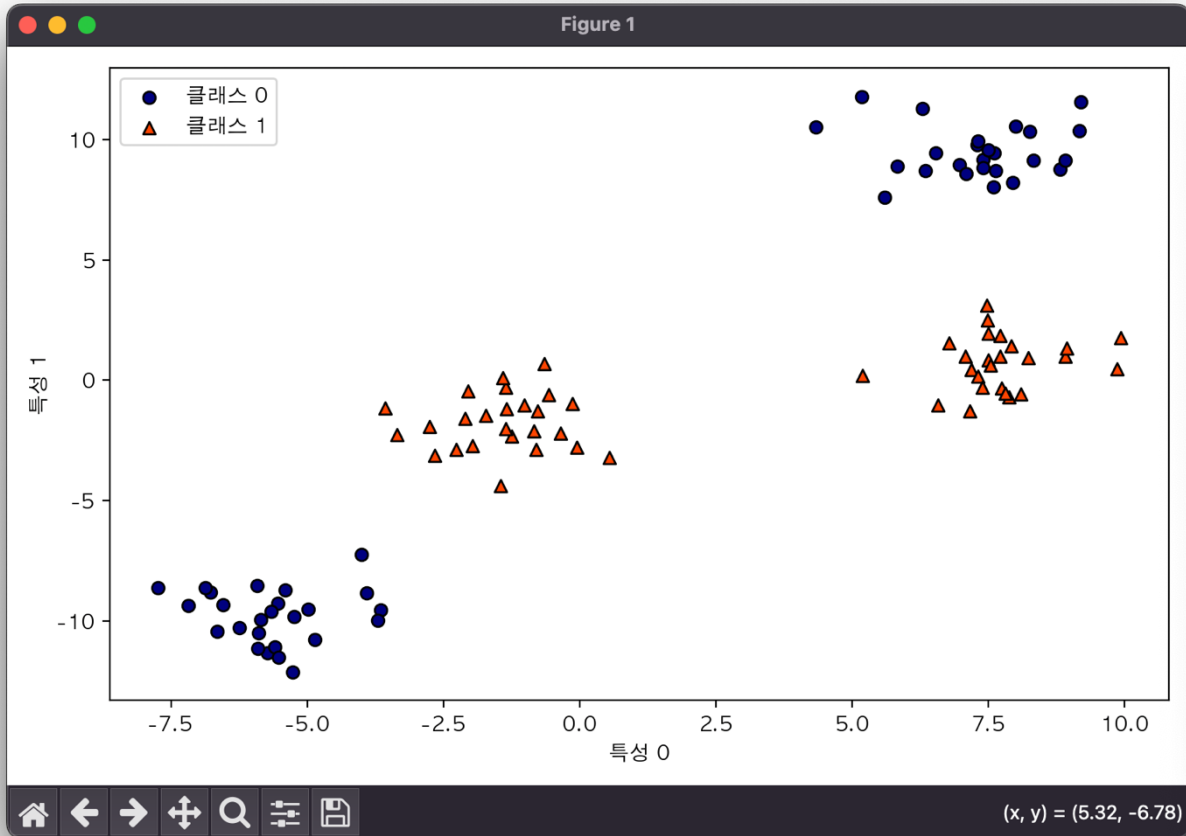


문제 4. 미생물 분류 - 코드

```
1 # 과정 10 - (문제4) 미생물 분류
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_blobs
4 from sklearn.model_selection import train_test_split
5 from sklearn.svm import SVC
6 from sklearn.neural_network import MLPClassifier
7 import matplotlib
8
9 matplotlib.rc('font', family='AppleGothic')
10 plt.rcParams['axes.unicode_minus'] = False
11
12 def main():
13     # 데이터 생성
14     x, y = make_blobs(centers=4, random_state=8)
15     y = y % 2
16
17     # 시각화
18     plt.figure(figsize=(8, 5))
19     for label, marker, color in zip([0, 1], ['o', '^'], ['navy', 'orangered']):
20         plt.scatter(
21             x[y == label, 0], x[y == label, 1],
22             marker=marker, color=color, edgecolor='black', label=f'클래스 {label}'
23         )
24     plt.xlabel('특성 0')
25     plt.ylabel('특성 1')
26     plt.legend()
27     plt.tight_layout()
28     plt.show()
```

```
1 # 데이터 분할
2 x_train, x_test, y_train, y_test = train_test_split(
3     x, y, random_state=0
4 )
5
6 # SVC 학습 및 평가 (C: -1, 0, 3 / gamma: -1 ~ 2)
7 print('[SVC 실험]')
8 for c in [-1, 0, 3]:
9     for g in [-1, 0, 1, 2]:
10         if c <= 0 or g < 0:
11             continue # SVC는 C > 0, gamma >= 0 필요
12         model = SVC(C=c, gamma=g)
13         model.fit(x_train, y_train)
14         train_score = model.score(x_train, y_train)
15         test_score = model.score(x_test, y_test)
16         print(f'C={c}, gamma={g} → 훈련 정확도: {train_score:.2f}, 테스트 정확도: {test_score:.2f}')
17
18 # [보너스 과제] MLPClassifier 사용
19 print('\n[MLPClassifier 실험]')
20 model = MLPClassifier(random_state=0, max_iter=1000)
21 model.fit(x_train, y_train)
22 train_score = model.score(x_train, y_train)
23 test_score = model.score(x_test, y_test)
24 print(f'MLP → 훈련 정확도: {train_score:.2f}, 테스트 정확도: {test_score:.2f}')
25
26
27 if __name__ == '__main__':
28     main()
```


문제 4. 미생물 분류 - 구현화면



```
swpilotPyhon5 — zsh — 99x28

[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process10/10-4/main.py ]
[SVC 실험]
C=3, gamma=0 → 훈련 정확도 : 0.52, 테스트 정확도 : 0.44
C=3, gamma=1 → 훈련 정확도 : 1.00, 테스트 정확도 : 1.00
C=3, gamma=2 → 훈련 정확도 : 1.00, 테스트 정확도 : 0.96

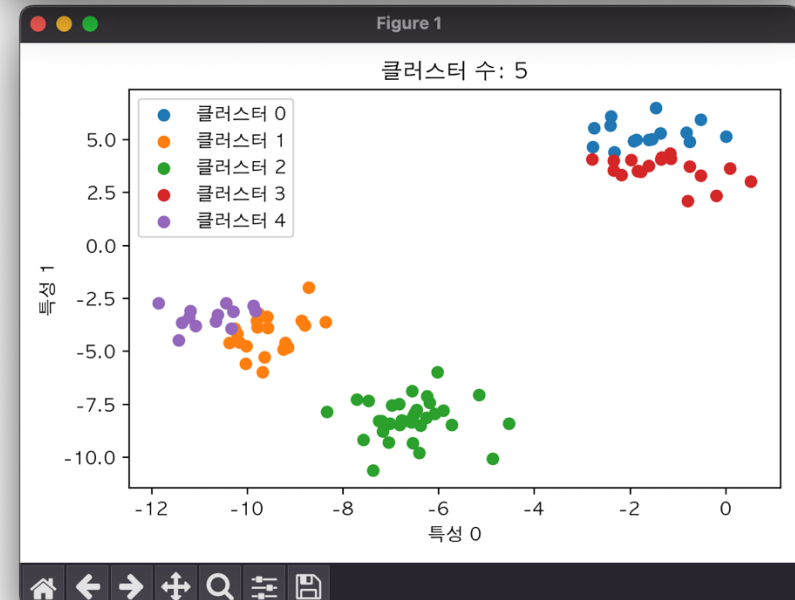
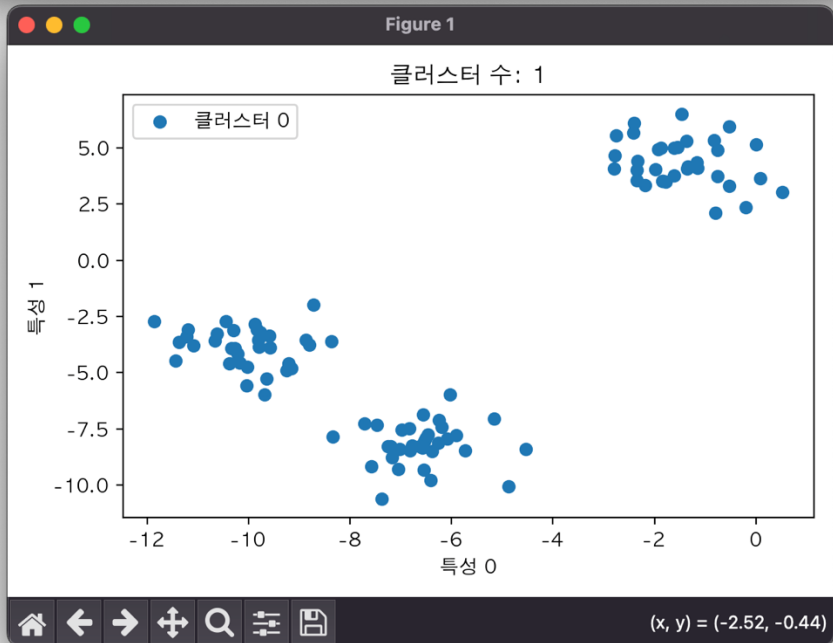
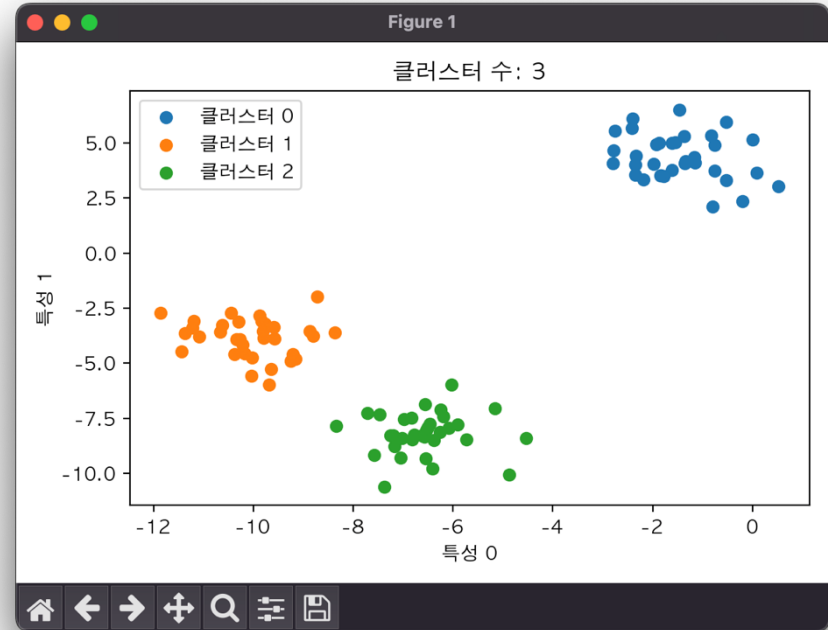
[MLPClassifier 실험]
MLP → 훈련 정확도 : 1.00, 테스트 정확도 : 1.00
(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```

문제 5. 이전에 없었던 물질을 분석하자 - 코드

```
1 # 과정 10 - (문제5) 이전에 없었던 물질을 분석하자
2 import matplotlib.pyplot as plt
3 import matplotlib
4 from sklearn.datasets import make_blobs
5 from sklearn.cluster import KMeans
6
7 matplotlib.rc('font', family='AppleGothic')
8 plt.rcParams['axes.unicode_minus'] = False
9
10 def main():
11     # 데이터 생성
12     x, y = make_blobs(random_state=1)
13
14     # 클러스터 수 3으로 설정
15     model = KMeans(n_clusters=3, random_state=0)
16     model.fit(x)
17     labels = model.labels_
18     print('클러스터 레이블:', labels)
19
20     # [보너스 과제] 클러스터 시각화 (n_clusters = 1, 3, 5)
21     for n_clusters in [1, 3, 5]:
22         model = KMeans(n_clusters=n_clusters, random_state=0)
23         model.fit(x)
24         labels = model.labels_
25
26         plt.figure(figsize=(6, 4))
27         plt.title(f'클러스터 수: {n_clusters}')
28         for cluster in range(n_clusters):
29             plt.scatter(
30                 x[labels == cluster, 0],
31                 x[labels == cluster, 1],
32                 label=f'클러스터 {cluster}'
33             )
34         plt.xlabel('특성 0')
35         plt.ylabel('특성 1')
36         plt.legend()
37         plt.tight_layout()
38         plt.show()
39
40
41 if __name__ == '__main__':
42     main()
```

문제 5. 이전에 없었던 물질을 분석하자 - 구현화면

```
swpilotPyhon5 — zsh — 99x28
((swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process10/10-5/main.py
클러스터 레이블: [0 1 1 1 2 2 2 1 0 0 1 1 2 0 2 2 2 0 1 1 2 1 2 0 1 2 2 0 0 2 0 0 2 0 1 2 1
1 1 2 2 1 0 1 1 2 0 0 0 0 1 2 2 2 0 2 1 1 0 0 1 2 2 1 1 2 0 2 0 1 1 1 2 0
0 1 2 2 0 1 0 1 1 2 0 0 0 0 1 0 2 0 0 1 1 2 2 0 2 0]
(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```



문제 6. 이전에 없었던 물질을 분석하자 II - 코드

```
1 # 과정 10 - (문제5) 이전에 없었던 물질을 분석하자 II
2 import matplotlib.pyplot as plt
3 import matplotlib
4 from sklearn.datasets import make_blobs
5 from sklearn.cluster import AgglomerativeClustering
6 from sklearn.metrics import silhouette_score
7 from scipy.cluster.hierarchy import dendrogram, linkage
8
9 matplotlib.rc('font', family='AppleGothic')
10 plt.rcParams['axes.unicode_minus'] = False
11
12 def main():
13     x, y = make_blobs(random_state=1)
14
15     # 병합 군집 (n_clusters=3)
16     model = AgglomerativeClustering(n_clusters=3)
17     labels = model.fit_predict(x)
18     print('클러스터 레이블:', labels)
19
20     # 덴드로그램 그리기 (기본 ward)
21     linked = linkage(x, method='ward')
22     plt.figure(figsize=(10, 5))
23     dendrogram(linked)
24     plt.title('덴드로그램 (linkage=ward)')
25     plt.xlabel('샘플 인덱스')
26     plt.ylabel('거리')
27     plt.tight_layout()
28     plt.show()
```

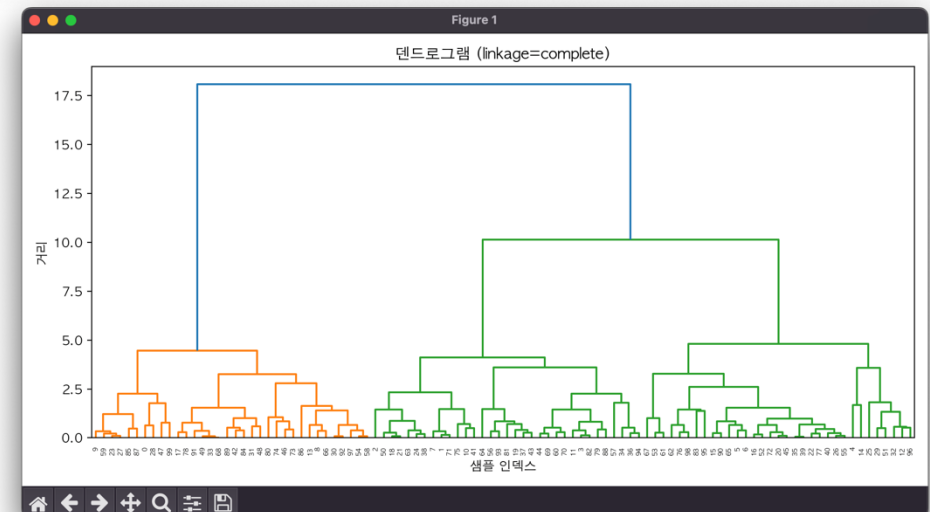
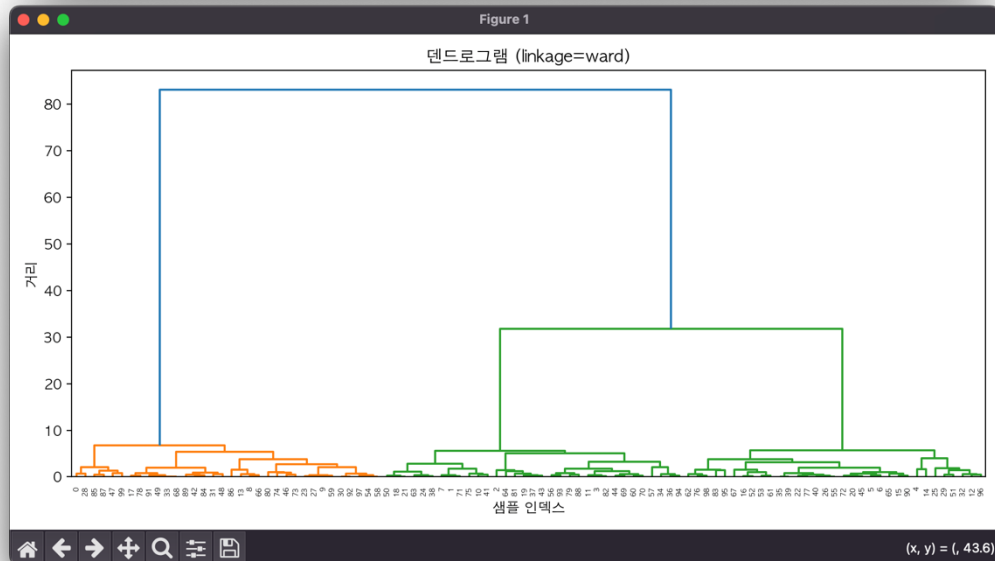
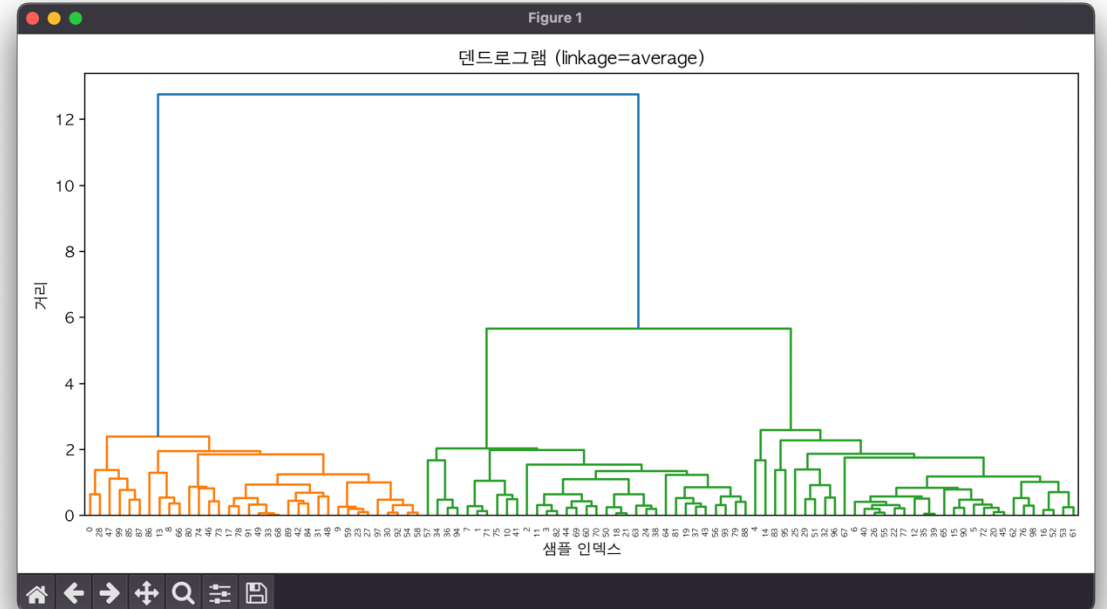
```
1 # [보너스 과제] linkage별 성능 비교 및 적합한 옵션 선택
2 best_method = None
3 best_score = -1
4 print('\n[linkage 옵션별 실루엣 점수 비교]')
5 for method in ['ward', 'average', 'complete']:
6     model = AgglomerativeClustering(n_clusters=3, linkage=method)
7     labels = model.fit_predict(x)
8     score = silhouette_score(x, labels)
9     print(f'{method}: 실루엣 점수 = {score:.4f}')
10    if score > best_score:
11        best_score = score
12        best_method = method
13
14    # 덴드로그램 시각화
15    linked = linkage(x, method=method)
16    plt.figure(figsize=(10, 5))
17    dendrogram(linked)
18    plt.title(f'덴드로그램 (linkage={method})')
19    plt.xlabel('샘플 인덱스')
20    plt.ylabel('거리')
21    plt.tight_layout()
22    plt.show()
23
24    print(f'\n가장 적합한 linkage 방식: {best_method} (실루엣 점수: {best_score:.4f})')
25
26
27 if __name__ == '__main__':
28     main()
```

문제 6. 이전에 없었던 물질을 분석하자 II - 구현화면

```
swpilotPyhon5 --zsh -- 99x28
[swpilot] hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process10/10-6/main.py
클러스터 레이블: [0 2 2 2 1 1 1 2 0 0 2 2 1 0 1 1 1 0 2 2 1 2 1 0 2 1 1 0 0 1 0 0 1 0 2 1 2
2 2 1 1 2 0 2 2 1 0 0 0 0 2 1 1 1 0 1 2 2 0 0 2 1 1 2 2 1 0 1 0 2 2 2 1 0
0 2 1 1 0 2 0 2 2 1 0 0 0 0 2 0 1 0 0 2 2 1 1 0 1 0]

[linkage 옵션별 실루엣 점수 비교]
ward: 실루엣 점수 = 0.7697
average: 실루엣 점수 = 0.7697
complete: 실루엣 점수 = 0.7697

가장 적합한 linkage 방식: ward (실루엣 점수: 0.7697)
[swpilot] hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```



문제 7. 교차 검증 - 코드

```
1 # 과정 10 - (문제7) 교차 검증
2 from sklearn.datasets import load_iris
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.model_selection import cross_val_score, train_test_split
5
6 def main():
7     # 데이터 로드
8     iris = load_iris()
9     x = iris.data
10    y = iris.target
11
12    # 로지스틱 회귀 모델
13    model = LogisticRegression(max_iter=1000)
14
15    # 교차 검증 (기본 3-Fold)
16    scores_3 = cross_val_score(model, x, y, cv=3)
17    print('3-Fold 교차 검증 결과:', scores_3)
18    print('3-Fold 평균 정확도:', scores_3.mean())
19
20    # 교차 검증 (5-Fold)
21    scores_5 = cross_val_score(model, x, y, cv=5)
22    print('\n5-Fold 교차 검증 결과:', scores_5)
23    print('5-Fold 평균 정확도:', scores_5.mean())
24
25    # [보너스 과제] train_test_split과 비교
26    x_train, x_test, y_train, y_test = train_test_split(
27        x, y, test_size=0.25, random_state=0
28    )
29    model.fit(x_train, y_train)
30    score_split = model.score(x_test, y_test)
31    print('\ntrain_test_split 테스트 정확도:', score_split)
32
33
34 if __name__ == '__main__':
35     main()
```

문제 7. 교차 검증 - 구현화면

```
swpilotPyhon5 — -zsh — 99x28
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process10/10-7/main.py ]
3-Fold 교차 검증 결과 : [0.98 0.96 0.98]
3-Fold 평균 정확도 : 0.9733333333333333

5-Fold 교차 검증 결과 : [0.96666667 1.          0.93333333 0.96666667 1.          ]
5-Fold 평균 정확도 : 0.9733333333333334

train_test_split 테스트 정확도 : 0.9736842105263158
(swipilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```


문제 8. 집으로 가는 길 - 코드

```
1 # 과정 10 - (문제8) 집으로 가는 길
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.neighbors import KNeighborsClassifier
5 import joblib
6
7 def main():
8     iris = load_iris()
9     x = iris.data
10    y = iris.target
11
12    x_train, x_test, y_train, y_test = train_test_split(
13        x, y, test_size=0.25, random_state=0
14    )
15
16    model = KNeighborsClassifier(n_neighbors=3)
17    model.fit(x_train, y_train)
18
19    accuracy = model.score(x_test, y_test)
20    print('테스트 정확도:', accuracy)
21
22    joblib.dump(model, 'iris_model.pkl')
23
24
25 if __name__ == '__main__':
26     main()
```

```
1 # 과정 10 - (문제8) 집으로 가는 길
2 from fastapi import FastAPI
3 from pydantic import BaseModel
4 import joblib
5 import numpy as np
6
7 app = FastAPI()
8 model = joblib.load('iris_model.pkl')
9
10 class IrisInput(BaseModel):
11     features: list[float]
12
13 @app.post('/predict')
14 def predict_iris(data: IrisInput):
15     x_input = np.array(data.features).reshape(1, -1)
16     prediction = model.predict(x_input)
17     return {'prediction': int(prediction[0])}
```

```
1 # 과정 10 - (문제8) 집으로 가는 길
2 FROM python:3.11-slim
3
4 WORKDIR /app
5
6 COPY main.py .
7 COPY main_api.py .
8 COPY iris_model.pkl .
9 COPY requirements.txt .
10
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 CMD ["uvicorn", "main_api:app", "--host", "0.0.0.0", "--port", "8000"]
```


문제 8. 집으로 가는 길 - 구현화면

```
10-8 - zsh - 99x28
(swipilot) hwangmungyu@hwangmungyuui-MacBookPro 10-8 % python main.py
테스트 정확도: 0.9736842105263158
(swipilot) hwangmungyu@hwangmungyuui-MacBookPro 10-8 %
```

```
10-8 - docker run -p 8000:8000 iris-fastapi - 99x28
[[swipilot] hwangmungyu@hwangmungyuui-MacBookPro 10-8 % docker run -p 8000:8000 iris-fastapi
/usr/local/lib/python3.11/site-packages/sklearn/base.py:440: InconsistentVersionWarning: Trying to
unpickle estimator KNeighborsClassifier from version 1.6.1 when using version 1.7.0. This might lea
d to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 192.168.65.1:53750 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.65.1:21891 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.65.1:63574 - "POST /predict HTTP/1.1" 200 OK
]]
```

```
hwangmungyu - zsh - 100x30
(base) hwangmungyu@hwangmungyuui-MacBookPro ~ % curl -X POST "http://localhost:8000/predict" \
-H "Content-Type: application/json" \
-d '{"features": [6.0, 2.9, 4.5, 1.5]}'
{"prediction":1}
(base) hwangmungyu@hwangmungyuui-MacBookPro ~ % curl -X POST "http://localhost:8000/predict" \
-H "Content-Type: application/json" \
-d '{"features": [6.7, 3.0, 5.2, 2.3]}'
{"prediction":2}
(base) hwangmungyu@hwangmungyuui-MacBookPro ~ % curl -X POST "http://localhost:8000/predict" \
-H "Content-Type: application/json" \
-d '{"features": [5.1, 3.5, 1.4, 0.2]}'
{"prediction":0}
(base) hwangmungyu@hwangmungyuui-MacBookPro ~ %
```